Conception de l'architecture applicative

Application mobile Android - choix d'un backend REST API

Architecture retenue : architecture en couches (Presentation Layer, Business Logic, Data Layer) avec pattern MVVM pour la couche de presentation. Le stockage principal est distant via une API REST, avec Room/SQLite comme cache local optionnel.

1. Architecture generale

L'application est concue selon une architecture en couches afin d'assurer une separation claire des responsabilites, une meilleure maintenabilite du code et une evolution plus simple du projet. La couche de presentation gere l'interaction avec l'utilisateur, la couche metier porte les regles de gestion, tandis que la couche d'acces aux donnees centralise la communication avec les sources locales et distantes.

Le choix retenu pour ce projet est un backend REST API. Ainsi, l'application Android joue le role de client mobile et communique avec le serveur a travers des requetes HTTP/JSON. Cette approche facilite la centralisation des donnees, la gestion multi-utilisateur, la securite, ainsi que l'evolutivite de la solution.

2. Couches principales

Couche	Composants principaux	Responsabilites
Presentation Layer	Ecrans, ViewModel, UI State, navigation	Affichage, collecte des actions utilisateur, observation des donnees et mise a jour de l'interface.
Business Logic	Use Cases, services metier, validateurs	Traitement des regles de gestion : authentification, courriers, OCR, classification, notifications.
Data Layer	Repositories, API service, Room DAO, DTO	Acces aux donnees, synchronisation, cache local, communication avec le backend REST.

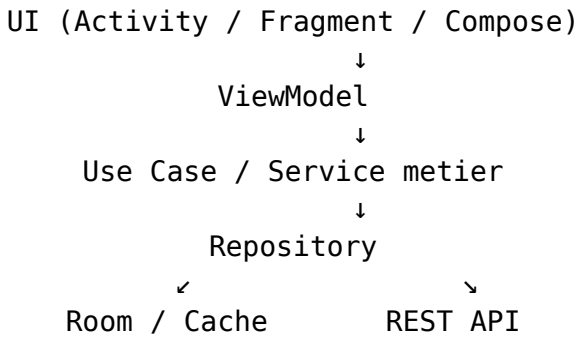
3. Adoption du pattern MVVM

Dans la couche de presentation, l'architecture MVVM est adoptee pour separer la vue de la logique de presentation. La View correspond aux ecrans Android (Activity, Fragment ou Compose), le ViewModel gere l'etat de l'interface et orchestre les actions utilisateur, tandis que le Model represente les donnees manipulees par l'application.

Le flux principal est le suivant : l'utilisateur interagit avec l'ecran, l'action est transmise au ViewModel, le ViewModel invoque un Use Case de la couche metier, puis le Use Case sollicite

un Repository dans la couche Data. Enfin, le resultat remonte vers le ViewModel puis vers l'interface utilisateur.

Schema de circulation des appels



4. Composants principaux de l'application

- **Interface utilisateur :** ecran de connexion, ecran d'accueil, liste des courriers entrants et sortants, detail d'un courrier, formulaire de creation/modification, recherche et filtrage.
- **Logique metier :** authentification, gestion des courriers, gestion des statuts, OCR, classification, notifications, controle des roles et droits d'acces.
- **Acces aux donnees :** repositories, services Retrofit/Ktor pour l'API REST, base locale Room, DAO, DTO, gestion des pieces jointes et du texte OCR.

5. Interactions entre les couches

- L'utilisateur agit sur l'interface utilisateur.
- La vue transmet l'action au ViewModel.
- Le ViewModel appelle un Use Case de la couche metier.
- Le Use Case utilise un Repository de la couche Data.
- Le Repository communique avec l'API REST pour le stockage principal et peut utiliser Room pour le cache local.
- La reponse remonte vers le ViewModel, qui met a jour l'etat de l'interface.

6. Choix du stockage et modularisation Gradle

Le mode de stockage principal choisi est un stockage distant via REST API. Les donnees metier sont centralisees dans le backend, ce qui permet de mieux gerer l'authentification, les roles, la synchronisation et la persistance des courriers. Une base locale Room/SQLite peut etre conservee pour le cache, le prechargement et un mode hors ligne limite.

Pour renforcer la separation entre les couches, le projet peut etre organise en modules Gradle distincts. Cette modularisation reduit le couplage, facilite les tests et rend l'application plus maintenable.

Module Gradle	Contenu
:app	Point d'entree Android et assemblage final

:presentation	UI, ViewModel, navigation, UI State
:domain	Use Cases, modeles metier, interfaces Repository
:data	Repositories, API REST, Room, DTO, DAO
:core	utilitaires communs, constantes, wrappers de resultat

7. Version UML / PlantUML

```
@startuml
title Architecture applicative Android - MVVM + REST API
actor Utilisateur
package Presentation {
  [UI]
  [ViewModel]
}
package Business {
  [Use Cases]
  [Services metier]
}
package Data {
  [Repositories]
  [Room / Cache]
  [REST API]
}
Utilisateur --> [UI]
[UI] --> [ViewModel]
[ViewModel] --> [Use Cases]
[Use Cases] --> [Services metier]
[Services metier] --> [Repositories]
[Repositories] --> [REST API]
[Repositories] --> [Room / Cache]
@enduml
```

ملاحظة مختصرة

في هذا المشروع تم اختيار REST API كخيار أساسي في الخلفية، مع اعتماد MVVM في الواجهة وتنظيم التطبيق على شكل طبقات واضحة: Presentation و Business Logic و Data Layer. هذا الاختيار يسهل الصيانة، الاختبار، وتطوير التطبيق لاحقا.

Conclusion

L'architecture proposee repond aux besoins du projet en combinant une application Android modulaire, un pattern MVVM pour la presentation, une couche metier claire et une couche Data connectee a un backend REST API. Cette conception offre une base solide pour la realisation, les tests et l'evolution future de la solution.